

## Task 1 — Assertion Generation

**What the papers say:** ChIRAAG says format the spec into structured sections first (Introduction, Signals, Functional Requirements, Timing Requirements). LAAG-RV says then add the RTL port declaration once so signal names are exact. Never give the full implementation body to the LLM.

### What you actually do:

For each of the 4 designs, you create a structured spec document manually. Here is the exact template you fill in:

=== DESIGN NAME ===

Introduction:

[1-2 sentences: what does this module do?]

Signals:

- signal\_name : bit-width, what it does

Functional Requirements:

1. [rule 1 about what must happen]
  2. [rule 2 about what must never happen]
- ...

Timing Requirements:

- Clock edge, reset behavior, output delay

=== PORT DECLARATION (for signal sync) ===

module name ( ...ports... );

You send this to Claude. Claude generates 8-12 raw SVA assertions. You save them. You save every prompt you sent and every response you got — that is your prompt log deliverable.

---

## Task 2 — Formalization

**What the papers say:** ChIRAAG found 27% of raw LLM assertions had errors — syntax errors and missing signals were the top two. LAAG-RV fixed missing signals by syncing signal names upfront. Both papers classified assertions into functional categories.

### What you actually do:

Go through every assertion the LLM gave you and fill in this table:

| Assertion    | Type      | Syntax OK?         | Signal Names OK? | Intent                             |
|--------------|-----------|--------------------|------------------|------------------------------------|
| p_fifo_full  | Safety    | Yes                | Yes              | full never asserted when count < 8 |
| p_fifo_empty | Invariant | No — missing rst_n | Fixed            | empty true after reset             |

For every assertion with a syntax error, paste it back to Claude with the error message and ask it to fix — exactly what ChIRAAG's Algorithm 1 Step 17 describes. That fixed assertion replaces the broken one in your file.

---

## Task 3 — Formal Validation

**What the papers say:** ChIRAAG used VCS simulator. SANGAM used JasperGold for formal verification. Both fed the error logs back to the LLM to fix assertions. This feedback loop is the core contribution of ChIRAAG.

### What you actually do:

Install SymbiYosys (free, open source, same principle as JasperGold). Run your assertions against your RTL. For every FAIL you get a counterexample — a specific input sequence that breaks the assertion. You then do exactly what ChIRAAG's paper describes:

Counterexample from SymbiYosys:  
Step 0: wr\_en=1, rd\_en=0, rst\_n=0  
Step 1: wr\_en=1, rd\_en=0, count=0  
Step 2: assertion p\_fifo\_count FAILED

- Paste this to Claude as Prompt 3
- Claude fixes the assertion
- Re-run SymbiYosys
- Repeat until PASS

Count your iterations. ChIRAAG needed 12 iterations for RV Timer. You will need similar. Record this number — it is a result you can compare against the paper.

---

## Task 4 — Critical Analysis

**What the papers say:** SANGAM's paper explicitly criticised AssertLLM for generating only width-check assertions (trivial). ChIRAAG showed assertions can detect real implementation bugs (the 4-bit adder with xor replaced by or). SANGAM achieved 74% property coverage and 90%+ branch coverage. These are your benchmarks.

**What you actually do:**

Answer these three questions the problem statement requires, using your results as evidence:

**Question 1 — Which assertions are correct but trivial?** These are assertions that always pass on any reasonable RTL, like checking that a 4-bit counter never exceeds 15. They are not useful for bug finding. You label them trivial in your table.

**Question 2 — Which are incorrect or over-restrictive?** These are assertions that fail even on a correct design because the LLM was too strict. Example: asserting that data\_out must change every single cycle — that is wrong. Explain what the LLM misunderstood.

**Question 3 — Which reveal bugs or corner cases?** Intentionally introduce a small bug into your RTL — like ChIRAAG did with the adder. For example in FIFO, make count increment even when rst\_n is low. Run your assertions. A good assertion will catch it. Report which one caught it and why that assertion is valuable.

---

## The Bonus Extension — Why It Makes Your Project Stand Out

Run Task 1 three times on the same design with different prompt inputs:

- Round 1: Structured spec only (ChIRAAG style)
- Round 2: Structured spec + port declaration (LAAG-RV style)
- Round 3: Structured spec + port declaration + waveform behavior description (SANGAM style)

Count how many assertions each round generates, how many pass formal verification, and what types they are. This comparison IS the comparison between the three papers, done on your own designs. Your table becomes original research data.

---

# THE 3C PART

## C1 — Correctness (SymbiYosys does this automatically)

You write your FIFO RTL correctly. You run SymbiYosys. If an assertion passes on correct RTL, it is correct. If it fails, the LLM was wrong about what the design does. You then feed the counterexample back to Claude and fix it. Your correctness score is:

Correctness = assertions that PASS on correct RTL / total assertions

ChIRaAG got 73% correct on first attempt (27% had errors). You report your percentage the same way.

---

## C2 — Consistency (Claude does this, then SymbiYosys confirms)

You send this specific prompt to Claude after getting all your assertions:

Here are all the assertions I generated for my FIFO:

[paste all assertions]

And here are the functional requirements from the spec:

[paste your spec requirements]

Please check:

1. Do any two assertions contradict each other?
2. Does any assertion contradict the spec?
3. Are there any impossible conditions where two assertions could never both be satisfied?

List every inconsistency you find.

Claude flags contradictions. You fix them. SymbiYosys then confirms — if two assertions genuinely contradict, one will always fail no matter what RTL you give it. That is your consistency check evidence.

---

## C3 — Completeness (you build a coverage matrix)

This is the most important one and also the simplest to do. You make a table like this for each design:

| Functional Requirement               | Assertion covering it   | Type      |
|--------------------------------------|-------------------------|-----------|
| full must assert when count = MAX    | p_full_when_max         | Safety    |
| write blocked when full              | p_no_write_when_full    | Safety    |
| empty after reset                    | p_empty_after_reset     | Invariant |
| count increments on write            | p_count_increment       | Invariant |
| every write eventually readable      | p_liveness_read         | Liveness  |
| <b>full and empty never together</b> | <b>NONE</b> ← gap found | —         |

Any row with NONE is a completeness gap. You then ask Claude to generate an assertion for that missing requirement. This is exactly what SANGAM's paper did — they ran signal-wise coverage to find which signals had no assertions yet.

For the formal coverage number, SymbiYosys gives you branch coverage and toggle coverage automatically in its output. SANGAM reported 90%+ branch coverage and 74% property coverage. You report the same metrics from your SymbiYosys output and compare.

---

## The key insight from the papers

LAAG-RV and ChIRAAG both say in their conclusions that they only achieved Correctness in their work, and left Consistency and Completeness as future work. SANGAM is the first to seriously address Completeness through signal-wise coverage. So in your Task 4 critical analysis, you can directly say: "consistent with ChIRAAG and LAAG-RV findings, our framework achieves high Correctness after iteration but Completeness remains a challenge" — and back it up with your coverage matrix numbers. That is a research-quality observation.